

Build a complete production-ready Full Stack User Registration module for my Spring Boot E-Commerce application.

Generate the backend completely first, then generate the frontend and integrate it with the generated REST APIs.

Create two separate projects:

- backend/ (Spring Boot)
- frontend/ (React + Vite)

Backend Requirements

Technology Stack

- Java
- Spring Boot 3
- Spring Security 6
- Spring Data JPA (Hibernate)
- MySQL 8
- JWT Authentication
- Maven
- Lombok
- Bean Validation
- Swagger/OpenAPI
- SLF4J Logging

- Flyway (or Liquibase)
- HikariCP
- MapStruct (preferred)

Implement the following REST endpoint:

POST /api/auth/register

Registration Request Fields

- Full Name
- Email Address
- Mobile Number
- Password
- Confirm Password

Validation Requirements

Full Name

- Required
- 3–100 characters
- Only alphabets and spaces

Email

- Required
- Valid email format
- Convert to lowercase

- Must be unique

Mobile Number

- Required
- Exactly 10 digits
- Numbers only
- Must be unique

Password

- Required
- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character

Confirm Password

- Must match Password

Registration Flow

- Validate request
- Check duplicate email
- Check duplicate mobile number
- Encrypt password using BCryptPasswordEncoder

- Save user in MySQL
- Generate JWT Access Token (15 minutes)
- Generate JWT Refresh Token (7 days)
- Return user details (excluding password) along with both tokens

JWT Requirements

- Stateless Authentication
- JWT should contain:
 - userId
 - email
 - role

Generate

- User Entity
- RegistrationRequest DTO
- RegistrationResponse DTO
- UserRepository
- AuthService
- AuthController
- JwtService
- JwtAuthenticationFilter

- SecurityConfig
- Password Encoder Configuration
- Global Exception Handler (@ControllerAdvice)
- Custom Exceptions
 - Duplicate Email
 - Duplicate Mobile Number
 - Invalid Password
 - Password Mismatch

Database Requirements

- Email UNIQUE
- Mobile UNIQUE
- Encrypted Password
- createdAt
- updatedAt

Return standardized JSON responses for success and failure.

Follow Spring Boot best practices, SOLID principles, Clean Architecture, and production-ready coding standards.

Frontend Requirements

Technology Stack

- React 19
- Vite
- React Router DOM
- Axios
- Bootstrap 5
- React Hook Form
- React Toastify
- React Icons
- Context API

Build only the React frontend.

The frontend must consume the REST APIs generated by the backend.

Implement

- Registration Page
- Authentication Context
- JWT Access & Refresh Token Handling
- Axios Instance
- Axios Request & Response Interceptors
- Protected Routes

- Client-side Form Validation
- Loading States
- Error Handling
- Toast Notifications
- Responsive UI (Mobile, Tablet, Desktop)

Registration page validation must exactly match the backend validation rules.

Use

- Reusable Components
- Reusable Services
- Environment Variables (.env)
- Clean Folder Structure
- React Best Practices
- Component-Based Architecture
- Responsive Bootstrap UI

Database & Environment Constraints

Use an existing local MySQL server.

I will provide the MySQL username, password, and database name when required.

Do NOT:

- Install MySQL.
- Create a new MySQL server or instance.
- Modify MySQL server settings.
- Change the MySQL port.
- Create additional MySQL users.
- Change the root password.
- Execute OS-level MySQL installation or configuration commands.
- Use Docker or Docker Compose.
- Generate Dockerfiles, docker-compose.yml, Kubernetes manifests, or container configurations.
- Run the application inside containers.
- Assume any MySQL credentials.

Your responsibility is only to:

- Connect to the existing MySQL server using the credentials I provide.
- Use the existing MySQL port (3306 unless I specify otherwise).

- Create the required database if it does not already exist.
- Create the required tables using Flyway (or Hibernate if Flyway is disabled).
- Configure application.properties (or application.yml) with the provided credentials.
- Verify the database connection.
- Use only the application database and do not modify any other databases.

General Constraints

- Never modify my local machine configuration.
- Never install or uninstall any software.
- Never modify environment variables outside the project.
- Never change system settings.
- Never use Docker, Docker Compose, Podman, Kubernetes, or any containerization technology.
- Do not generate container-related files unless I explicitly request them.
- Ask me for any required credentials (such as MySQL username, password, or database name) instead of assuming them.
- Generate only the application code and project configuration required to run the application locally.